

Nafil Fadillah
20123958
3KB02

Pertemuan Minggu Ke 8

```
1  male(lennart).
2  male(mike).
3  male(donald).
4  male(usain).
5  male(joar).
6  male(adam).
7  male(dan).
8  male(simon).
9  female(hillary).
10 female(elise).
11 female(lisa).
12 female(lena).
13
14 parent(mike, lennart).
15 parent(mike, lena).
16 parent(lennart, donald).
17 parent(lennart, hillary).
18 parent(lennart, usain).
19 parent(lena, adam).
20 parent(lena, simon).
21 parent(adam, dan).
22 parent(donald, lisa).
23 parent(hillary, joar).
24 parent(hillary, elise).
25
26
27 child(lisa).
28 child(joar).
29 child(elise).
30 child(dan).
31 child(simon).
32
33 %% predicate rules
34 father(X,Y) :- male(X),parent(X,Y).
35 mother(X,Y) :- female(X),parent(X,Y).
36 son(X,Y) :- male(X),parent(Y,X).
37 daughter(X,Y) :- female(X),parent(Y,X).
38
39 family_children(X, X):-
40     child(X).
41
42 family_children(X, Child):-
43     parent(X,Y),
44     family_children(Y, Child).
```

Penjelasan Kode :

Program yang diberikan ditulis menggunakan bahasa Prolog, yaitu bahasa pemrograman berbasis logika yang digunakan untuk merepresentasikan pengetahuan serta menarik kesimpulan berdasarkan fakta dan aturan. Dalam program ini, dibangun sebuah representasi silsilah keluarga (family tree) yang terdiri dari fakta dasar dan aturan hubungan antarindividu.

1. Fakta Dasar

Bagian awal program berisi kumpulan fakta yang menjadi fondasi utama sistem. Fakta-fakta ini merepresentasikan informasi yang dianggap benar tanpa perlu pembuktian lebih lanjut.

Pertama, terdapat pengelompokan individu berdasarkan jenis kelamin, yaitu laki-laki (*male*) dan perempuan (*female*). Misalnya, Lennart, Mike, dan Donald dikategorikan sebagai laki-laki, sedangkan Hillary, Elise, Lisa, dan Lena sebagai perempuan. Informasi ini penting karena akan digunakan dalam penentuan peran seperti ayah, ibu, anak laki-laki, dan anak perempuan.

Selanjutnya, terdapat fakta hubungan orang tua (*parent*) yang mendefinisikan keterkaitan antarindividu dalam keluarga. Format penulisannya adalah `parent(OrangTua, Anak)`. Contohnya, `parent(mike, lennart)` menunjukkan bahwa Mike adalah orang tua dari Lennart, sedangkan `parent(lennart, donald)` berarti Lennart adalah orang tua dari Donald.

Selain itu, terdapat fakta *child* yang secara eksplisit menyatakan beberapa individu sebagai anak, seperti Lisa, Joar, Elise, Dan, dan Simon. Fakta ini berperan sebagai titik awal dalam proses pencarian keturunan yang lebih kompleks.

2. Aturan Predikat

Setelah fakta dasar, program mendefinisikan sejumlah aturan (*rules*) untuk membangun hubungan keluarga yang lebih spesifik. Aturan ini memungkinkan sistem menarik kesimpulan baru berdasarkan fakta yang sudah ada.

Predikat $\text{father}(X, Y)$ menyatakan bahwa X adalah ayah dari Y jika X berjenis kelamin laki-laki dan merupakan orang tua dari Y. Sebaliknya, $\text{mother}(X, Y)$ menyatakan bahwa X adalah ibu dari Y jika X berjenis kelamin perempuan dan merupakan orang tua dari Y.

Selain itu, terdapat aturan untuk hubungan anak, yaitu $\text{son}(X, Y)$ dan $\text{daughter}(X, Y)$. Predikat $\text{son}(X, Y)$ berarti X adalah anak laki-laki dari Y jika X berjenis kelamin laki-laki dan Y adalah orang tua X. Sedangkan $\text{daughter}(X, Y)$ menyatakan bahwa X adalah anak perempuan dari Y jika X berjenis kelamin perempuan dan Y adalah orang tua X.

Dengan adanya aturan-aturan ini, sistem tidak hanya menyimpan data, tetapi juga mampu menyimpulkan hubungan keluarga secara otomatis.

3. Rekursi pada `family_children`

Bagian terakhir menggunakan konsep rekursi, yaitu teknik di mana suatu aturan memanggil dirinya sendiri untuk menyelesaikan masalah yang lebih kompleks.

Pada kasus dasar, seseorang dianggap sebagai keturunan jika ia terdaftar dalam fakta $\text{child}(X)$. Ini menjadi titik awal pencarian.

Kemudian, pada kasus rekursif, aturan menyatakan bahwa seorang individu adalah keturunan dari X jika terdapat hubungan berantai, yaitu X adalah orang tua dari Y, dan individu tersebut merupakan keturunan dari Y. Dengan kata lain, aturan ini menelusuri garis keturunan secara berulang hingga mencapai individu paling bawah.

Melalui mekanisme ini, program mampu menemukan tidak hanya anak langsung, tetapi juga keturunan yang lebih jauh seperti cucu, cicit, dan seterusnya.

4. Gambaran Silsilah Keluarga

Berdasarkan fakta yang ada, struktur keluarga dapat divisualisasikan secara sederhana. Mike berada pada generasi tertua sebagai leluhur. Ia memiliki anak Lennart dan Lena.

Dari Lennart, lahir beberapa anak seperti Donald, Hillary, dan Usain. Sementara itu, dari Lena lahir Adam dan Simon. Kemudian, Adam memiliki anak bernama Dan. Struktur ini menunjukkan hubungan bertingkat yang membentuk garis keturunan dari generasi ke generasi.

Secara keseluruhan, program ini menunjukkan bagaimana Prolog dapat digunakan untuk memodelkan hubungan keluarga secara logis dan sistematis, serta memungkinkan penelusuran hubungan yang kompleks melalui kombinasi fakta dan aturan rekursif.

Pertemuan VIII

```
# Python3 program to print the path from root node to destination node for (N * N - 1) puzzle algorithm using Branch and Bound
# The solution assumes that instance of puzzle is solvable

# Importing copy for deepcopy function
import copy

# Importing the heap functions from python library for Priority Queue
from heapq import heappush, heappop

# This variable can be changed to change the program from 8 puzzle(n=3) to 15 puzzle(n=4) to 24 puzzle(n=5), etc.
n = 3

# Bottom, left, top, right
row = [ 1, 0, -1, 0 ]
col = [ 0, -1, 0, 1 ]

# A class for Priority Queue
class priorityQueue:

    # Constructor to initialize a Priority Queue
    def __init__(self):
        self.heap = []

    # Inserts a new key 'k'
    def push(self, k):
        heappush(self.heap, k)

    # Method to remove minimum element from Priority Queue
    def pop(self):
        return heappop(self.heap)

    # Method to know if the Queue is empty
    def empty(self):
        if not self.heap:
            return True
        else:
            return False
```

```

# Node structure
class node:

    def __init__(self, parent, mat, empty_tile_pos,
                 cost, level):

        # Stores the parent node of the current node helps in tracing path when the answer is found
        self.parent = parent

        # Stores the matrix
        self.mat = mat

        # Stores the position at which the empty space tile exists in the matrix
        self.empty_tile_pos = empty_tile_pos

        # Stores the number of misplaced tiles
        self.cost = cost

        # Stores the number of moves so far
        self.level = level

    # This method is defined so that the priority queue is formed based on the cost variable of the objects
    def __lt__(self, nxt):
        return self.cost < nxt.cost

# Function to calculate the number of misplaced tiles ie. number of non-blank tiles not in their goal position

```

```

def calculateCost(mat, final) -> int:

    count = 0
    for i in range(n):
        for j in range(n):
            if ((mat[i][j]) and
                (mat[i][j] != final[i][j])):
                count += 1

    return count

def newNode(mat, empty_tile_pos, new_empty_tile_pos,
           level, parent, final) -> node:

    # Copy data from parent matrix to current matrix
    new_mat = copy.deepcopy(mat)

    # Move tile by 1 position
    x1 = empty_tile_pos[0]
    y1 = empty_tile_pos[1]
    x2 = new_empty_tile_pos[0]
    y2 = new_empty_tile_pos[1]
    new_mat[x1][y1], new_mat[x2][y2] = new_mat[x2][y2], new_mat[x1][y1]

    # Set number of misplaced tiles
    cost = calculateCost(new_mat, final)

    new_node = node(parent, new_mat, new_empty_tile_pos,
                    cost, level)
    return new_node

# Function to print the N x N matrix
def printMatrix(mat):

    for i in range(n):
        for j in range(n):
            print("%d " % (mat[i][j]), end = " ")

        print()

# Function to check if (x, y) is a valid matrix coordinate
def isSafe(x, y):

```

```

        return x >= 0 and x < n and y >= 0 and y < n

# Print path from root node to destination node
def printPath(root):

    if root == None:
        return

    printPath(root.parent)
    printMatrix(root.mat)
    print()

# Function to solve N*N - 1 puzzle algorithm using Branch and Bound.
# empty_tile_pos is the blank tile position in the initial state.
def solve(initial, empty_tile_pos, final):

    # Create a priority queue to store live nodes of search tree
    pq = priorityQueue()

    # Create the root node
    cost = calculateCost(initial, final)
    root = node(None, initial,
                empty_tile_pos, cost, 0)

    # Add root to list of live nodes
    pq.push(root)

    # Finds a live node with least cost, add its children to list of live nodes and finally deletes it from the list.
    while not pq.empty():

        # Find a live node with least estimated cost and delete it from the list of live nodes
        minimum = pq.pop()

        # If minimum is the answer node
        if minimum.cost == 0:

            # Print the path from root to destination
            printPath(minimum)
            return

        # Generate all possible children
        for i in range(n):
            new_tile_pos = [

```

```

        minimum.empty_tile_pos[0] + row[i],
        minimum.empty_tile_pos[1] + col[i], ]

    if isSafe(new_tile_pos[0], new_tile_pos[1]):

        # Create a child node
        child = newNode(minimum.mat,
                        minimum.empty_tile_pos,
                        new_tile_pos,
                        minimum.level + 1,
                        minimum, final,)

        # Add child to list of live nodes
        pq.push(child)

```

```

# Initial configuration
# Value 0 is used for empty space
initial = [ [ 1, 2, 3 ],
            [ 5, 6, 0 ],
            [ 7, 8, 4 ] ]

# Solvable Final configuration
# Value 0 is used for empty space
final = [ [ 1, 2, 3 ],
          [ 5, 8, 6 ],
          [ 0, 7, 4 ] ]

# Blank tile coordinates in initial configuration
empty_tile_pos = [ 1, 2 ]

# Function call to solve the puzzle
solve(initial, empty_tile_pos, final)

# This code is contributed by Kevin Joshi

```

Penjelasan Kode :

Program Python yang ditampilkan bertujuan untuk menyelesaikan teka-teki geser (sliding puzzle) berukuran $N \times N$, seperti 8-puzzle (3x3) atau 15-puzzle (4x4). Program ini menggunakan algoritma Branch and Bound, yaitu metode pencarian solusi yang memilih langkah terbaik berdasarkan nilai biaya (cost) terendah. Output dari program ini adalah urutan langkah dari kondisi awal hingga mencapai kondisi tujuan.

Pada bagian awal, program mengimpor library copy dan heapq. Library copy digunakan untuk menyalin matriks agar tidak saling memengaruhi, sedangkan heapq digunakan untuk membuat Priority Queue (antrian prioritas). Variabel n digunakan untuk menentukan ukuran puzzle, misalnya $n = 3$ untuk 8-puzzle. Selain itu, terdapat array row dan col yang berfungsi untuk menentukan arah pergerakan ubin kosong, yaitu bawah, kiri, atas, dan kanan.

Program kemudian membuat kelas `priorityQueue` yang berfungsi untuk menyimpan node-node yang akan diperiksa. Kelas ini memiliki metode `push` untuk menambahkan node ke dalam antrian dan `pop` untuk mengambil node dengan nilai `cost` paling kecil. Dengan menggunakan `heapq`, antrian ini secara otomatis akan mengurutkan node berdasarkan prioritasnya.

Selanjutnya, terdapat kelas `node` yang digunakan untuk merepresentasikan setiap kondisi puzzle. Di dalam kelas ini terdapat beberapa atribut penting, yaitu `parent` (menyimpan node sebelumnya untuk melacak jalur solusi), `mat` (menyimpan kondisi matriks saat ini), `empty_tile_pos` (posisi ubin kosong), `cost` (jumlah ubin yang salah posisi), dan `level` (jumlah langkah yang telah dilakukan). Kelas ini juga memiliki fungsi `lt` yang digunakan untuk membandingkan node berdasarkan nilai `cost` agar dapat diproses dengan benar oleh `Priority Queue`.

Beberapa fungsi pendukung juga disediakan. Fungsi `calculateCost` digunakan untuk menghitung jumlah ubin yang tidak berada pada posisi yang benar (kecuali ubin kosong). Fungsi `newNode` digunakan untuk membuat node baru setelah ubin digeser, sekaligus memperbarui matriks, posisi ubin kosong, nilai `cost`, dan `level`. Fungsi `isSafe` digunakan untuk memastikan bahwa pergerakan ubin tidak keluar dari batas matriks. Fungsi `printPath` digunakan untuk mencetak jalur solusi dari awal hingga akhir dengan cara rekursif.

Proses utama terdapat pada fungsi `solve`. Program akan memasukkan kondisi awal ke dalam `Priority Queue`, kemudian selama antrian tidak kosong, program akan mengambil node dengan `cost` terendah. Jika nilai `cost` sama dengan 0, berarti kondisi tujuan telah tercapai dan jalur solusi akan dicetak. Jika belum, program akan membuat node-node baru berdasarkan semua kemungkinan gerakan ubin kosong, lalu memasukkannya kembali ke dalam antrian untuk diproses.

Pada bagian akhir program, ditentukan kondisi awal (`initial state`) dan kondisi tujuan (`goal state`). Angka 0 digunakan untuk menunjukkan ubin kosong yang bisa digeser. Program akan mencari jalur terbaik dari kondisi awal ke kondisi tujuan dengan jumlah langkah seminimal mungkin.

Secara keseluruhan, program ini menggunakan kombinasi struktur data Priority Queue, representasi node, dan algoritma Branch and Bound untuk menyelesaikan masalah sliding puzzle secara efektif dan efisien.